

DSA League Backend Learning Notes

1. Why the seed file exists

The file `app/seed.py` exists to prepare the database with initial data so the application has a realistic starting state.

Its responsibilities are:

- create a few initial users

- create sample daily entries for those users

- avoid inserting duplicate data on repeated runs

This file is separate because seeding is a different concern from:

- request handling

- authentication

- database connection setup

- business logic

It acts as a bootstrap layer.

2. How this file connects to the rest of the backend

The flow is:

`app/main.py`

↓

startup event

↓

`app/db.py`

↓

SQLAlchemy engine and session

↓

`app/seed.py`

↓

`app/models.py`

↓

`app/auth.py`

↓

database

What happens at startup

1. FastAPI starts.
2. The startup event in main.py runs.
3. The database tables are created.
4. The seed function is called.
5. The seed function opens a database session.
6. It checks whether users already exist.
7. If not, it creates users and entries.
8. The transaction is committed.

This is how the backend transitions from "empty environment" to "usable application".

3. Step-by-step explanation of the seed file

Imports

```
import asyncio
```

This allows the file to run asynchronous database operations.

```
from sqlalchemy import select
```

This imports SQLAlchemy's query builder so the code can ask the database for records.

```
import app.db as db
```

This connects the file to the database layer.

```
from app.models import User, Role, DailyEntry, Platform, Pattern, Difficulty
```

These are the ORM classes that represent database tables.

```
from app.auth import get_password_hash
```

This ensures passwords are stored securely as hashes instead of plaintext.

4. The users list

The users list is a Python list of dictionaries.

Each dictionary contains the values for one user:

username

full_name

email
college
role

This structure is useful because it keeps initial test data in one place and makes it easy to change.

5. The async seed function

async def seed():

This function performs the database seeding work.

It is asynchronous because database work is usually non-blocking in modern Python backends.

6. Engine initialization

DB engine setup is performed by:

db.get_engine()

Why it matters:

SQLAlchemy needs an engine before a session can be created.

The engine is responsible for creating and managing connections.

7. Opening a database session

async with db.AsyncSessionLocal() as session:

This opens an asynchronous SQLAlchemy session.

A session is the unit of work in the ORM.

It lets the program:

track objects

batch changes

commit or roll back safely

8. Checking whether data already exists

result = await session.execute(select(User))

```
if result.scalars().first():  
    return
```

This makes the seed process idempotent.

If users already exist, the function stops early so it does not duplicate data.

This is a very important backend pattern.

9. Creating users

For each item in users:

```
user = User(  
    username=item["username"],  
    full_name=item["full_name"],  
    email=item["email"],  
    college=item["college"],  
    role=item["role"],  
    hashed_password=await get_password_hash("Password123!"),  
)  
  
session.add(user)
```

This creates Python objects that represent rows in the users table.

The password is hashed before storage.

The object is added to the session but not committed yet.

10. Committing the user inserts

```
await session.commit()
```

This finalizes the transaction.

Without commit, the inserts would not be truly saved.

This is one of the most important concepts in backend development:

add objects to a session

commit to persist them

11. Creating sample entries

The code then fetches the created users and creates a DailyEntry for each non-admin user.

Each entry includes:

- a user id
- a platform
- a problem number and name
- a difficulty level
- time spent
- whether the problem was solved
- notes
- a score
- approval status

This gives the app realistic sample activity data.

12. Why this matters for backend concepts

This file demonstrates several core ideas:

SQLAlchemy concepts

- Session
- ORM mapping
- transaction
- commit
- query execution

Python concepts

- async / await
- loops
- dictionaries
- functions

Security concepts

- password hashing
- never storing plaintext passwords

Application architecture concepts

- startup initialization
- separation of concerns
- idempotency

13. What happens internally

In Python

the async function runs as a coroutine
await pauses execution until the database operation completes
the loop iterates over seeded data

In SQLAlchemy

the User and DailyEntry objects are mapped to database tables
session.add marks them for persistence
commit writes them to the database

In the database

rows are inserted into tables
foreign key relationships are represented through user_id

In the application lifecycle

startup prepares the system
the app becomes ready for requests

14. Five-minute recap

The seed file exists to initialize the database with starter data. It uses SQLAlchemy's async session, creates ORM objects, hashes passwords, and commits changes. It also prevents duplicate data by checking whether users already exist.

15. Interview questions

1. Why is a seed script usually designed to be idempotent?
2. What is the difference between adding an object to a session and committing it?
3. Why should passwords be hashed before storage?

16. Debugging challenge

If the app starts but no sample users appear, which layer is most likely involved?

Possible causes:

- the seed function is not being called
- the session was not created correctly
- the commit failed
- the database path is wrong

17. Coding exercise

Try writing your own minimal seed script:

- define a list of users
- create a User ORM object for each one
- add them to a session
- commit the session
- make sure the script does not insert duplicates

18. Real-world analogy

This file is like opening a new store and stocking the shelves with sample products so the store looks ready for customers on day one.

19. Common beginner mistake

A common beginner mistake is assuming that `session.add` immediately saves data.

In reality, the data is staged first and only becomes permanent after `commit`.